Operating Systems

Abdelrahman Hosny

2026

Faculty of Computers and Information, Assiut University

Contents

01
File Systems

02
Processes

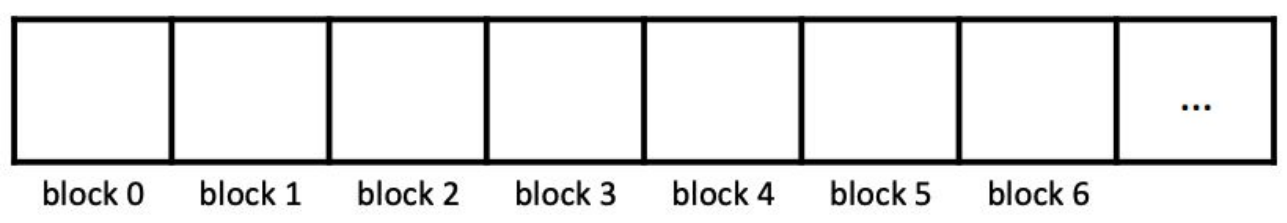
03
Threads

04
Virtual Memory

05
Miscellaneous

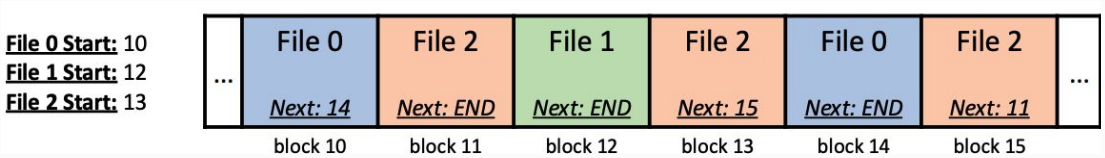
06
The Shell

Summary - Contiguous Allocation

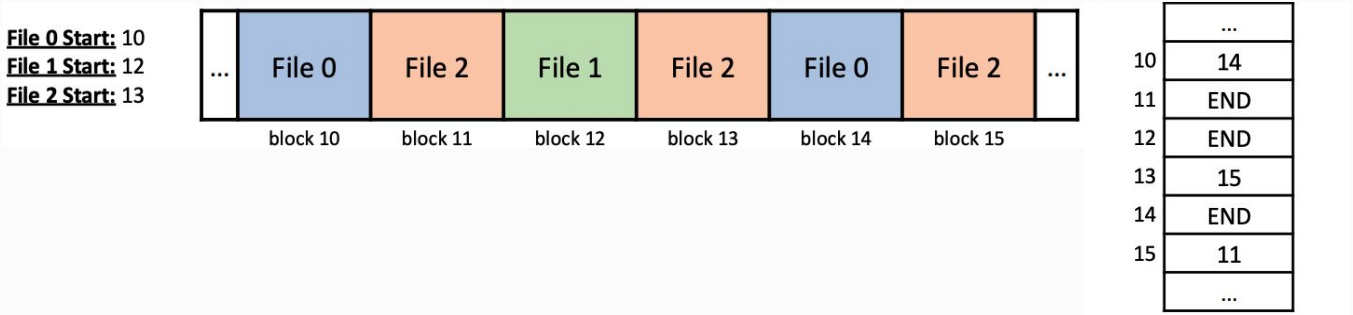


Summary - Non-contiguous Allocation

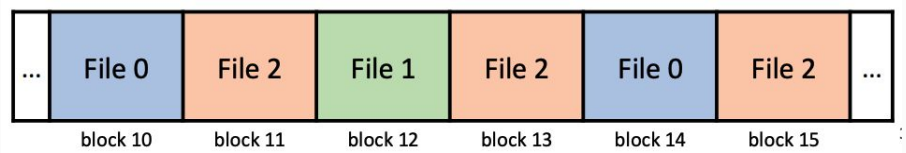
Files as linked lists



In-memory File Allocation Table



Multi-level Index



File 0: 10, 14

File 1: 12

File 2: 13, 15, 11

This Lecture

What is the multi-level index in a file system?

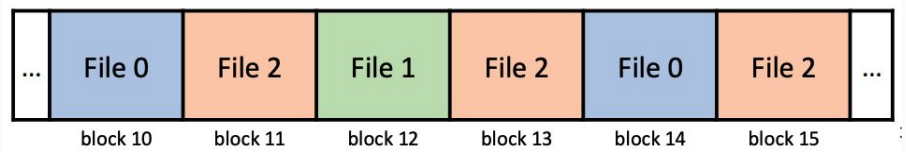
Unix v6 Filesystem

Multi-level Index

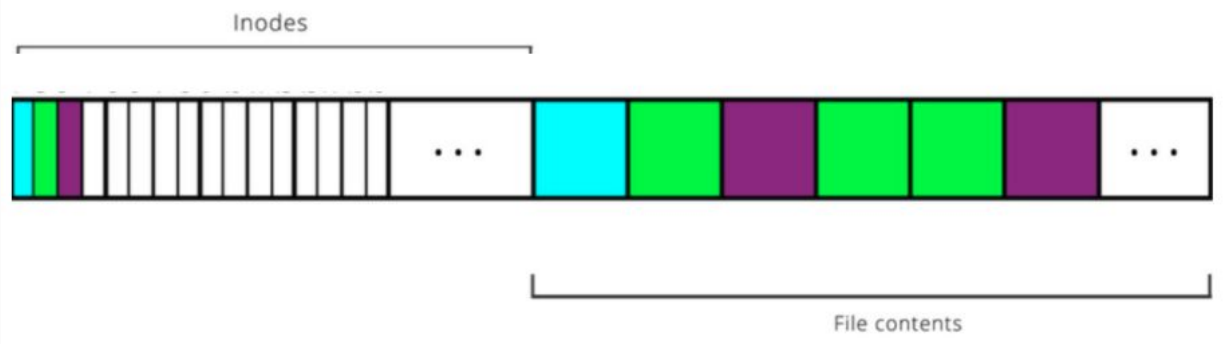
File 0: 10, 14

File 1: 12

File 2: 13, 15, 11



Let's reserve some space on disk to store this information for each file, separately from its payload data. This per-file space is called an **inode**.



inodes

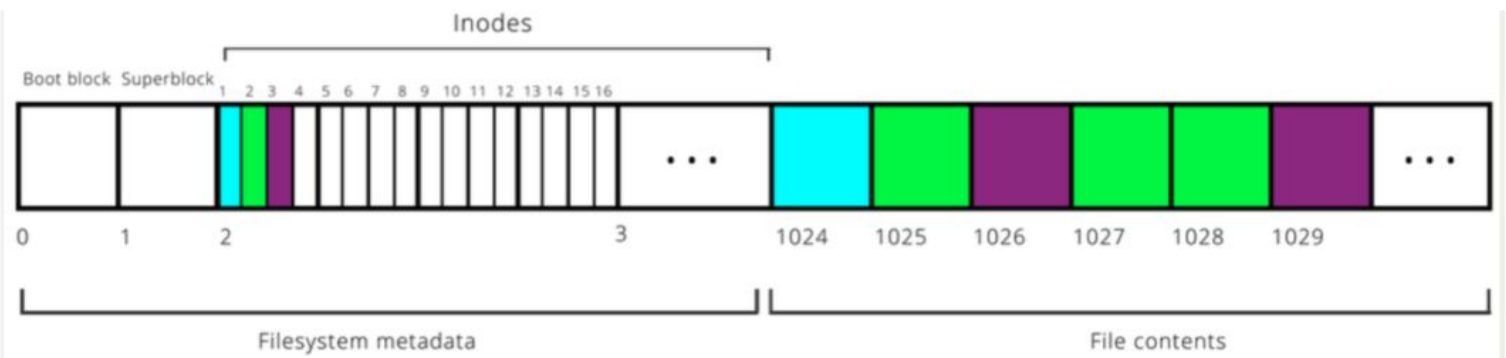
An **inode** ("index node") is a grouping of data about **a single file**, **stored on disk**.

- Various filesystems (e.g. Unix v6, contiguous allocation/linked files, but not FAT) store file metadata in inodes.
- In Unix v6, an inode contains **an ordered list** of block numbers that store the file's payload data, and also **stores other metadata like file size**.
- Unix v6 stores inodes **contiguously** in a reserved space called the **inode table**, which starts at block 2. (block 0 is "boot block" containing hard drive info, block 1 is "superblock" containing filesystem info).
- **Inodes** can be read into memory when used for quicker access

Unix v6 inodes

The Unix v6 filesystem stores inodes on disk together in the **inode table** for quick access.

- Unix v6 Inodes are 32 bytes big, and 1 block = 1 sector = 512 bytes, so 16 inodes/block.
- Typically, at most 10% of the drive stores metadata.
- Filesystem goes from **filename** to **inode number** ("inumber") to file data.



inodes

We need inodes to be a fixed size, and not too large. So how should we store the block numbers? How many should there be?

1. if variable number, there's no fixed inode size
2. if fixed number, this limits maximum file size

The inode design here has space for 8 block numbers, which are stored in order. (i.e. first block number stores first chunk of file, etc.). How do we support large files?

Unix V6 Inodes

Each inode stores file information and has space for 8 block numbers.

```
struct inode {
  uint16_t  i_mode;     // bit vector of file
                        //   type and permissions
  uint8_t   i_nlink;    // number of references
                        //   to file
  uint8_t   i_uid;      // owner
  uint8_t   i_gid;      // group of owner
  uint8_t   i_size0;    // most significant byte
                        //  of size
  uint16_t  i_size1;    // lower two bytes of size
                        //  (size is encoded in a
                        //  three-byte number)
  uint16_t  i_addr[8];  // device addresses
                        //   constituting file
  uint16_t  i_atime[2]; // access time
  uint16_t  i_mtime[2]; // modify time
};
```

https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/include/linux/fs.h?h=v6.19.6

Unix V6 Inodes

Each inode stores file information and has space for 8 block numbers.

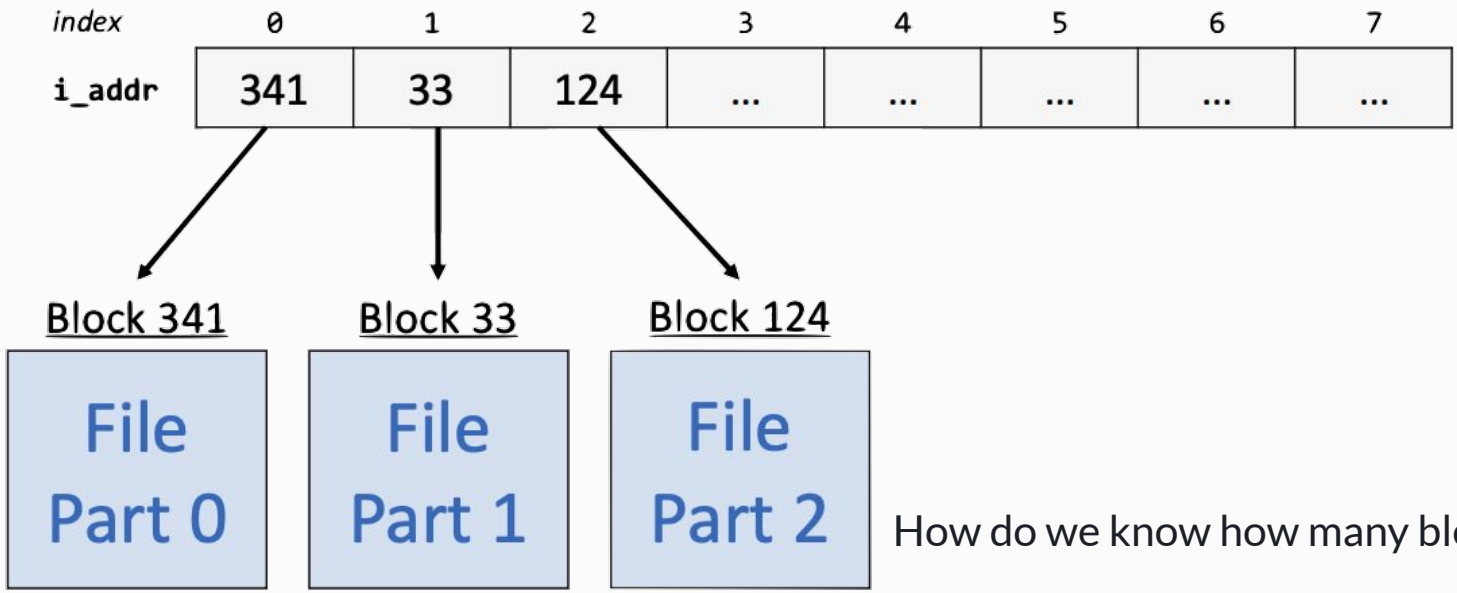
```
struct inode {
  uint16_t  i_mode;     // bit vector of file
                        //   type and permissions
  uint8_t   i_nlink;    // number of references
                        //   to file
  uint8_t   i_uid;      // owner
  uint8_t   i_gid;      // group of owner
  uint8_t   i_size0;    // most significant byte
                        //  of size
  uint16_t  i_size1;    // lower two bytes of size
                        //  (size is encoded in a
                        //  three-byte number)
  uint16_t  i_addr[8];  // device addresses
                        //   constituting file
  uint16_t  i_atime[2]; // access time
  uint16_t  i_mtime[2]; // modify time
};
```

https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/include/linux/fs.h?h=v6.19.6

For now, we just need i_addr; an array of 8 block numbers. i_addr entries are in file data order, not necessarily numerical order - the blocks could be scattered all over disk. E.g. a file could have i_addr = [12, 200, 56, ...]. It uses only as many i_addr entries as needed to store file data

Unix V6 Inodes

i_addr stores the numbers of blocks that contain payload data.



How do we know how many blocks we are using?

Practice # 1

Design: 1 block = 1 sector = 512 bytes

Let's say we have an inode that looks like the following:

Inode:

size = 600 bytes

i_addr = [122, 56, ….]

1. How many bytes of block 122 store file payload data?
2. How many bytes of block 56 store file payload data?

Practice # 2

Design: 1 block = 1 sector = 512 bytes

Let's say we have an inode that looks like the following:

Inode:

size = 2000 bytes

i_addr = [56, 122, 45, 22, …]

1. Which block number stores the index-1500th byte (0-indexed) of the file?

Large files

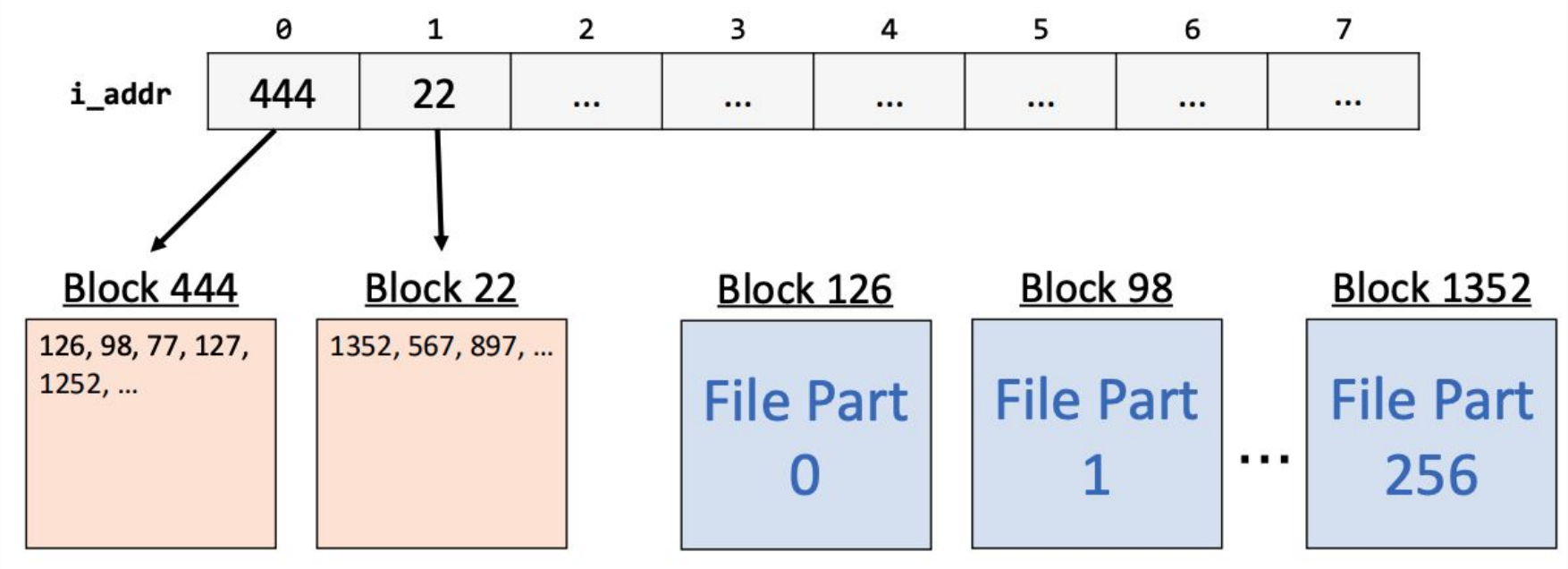
Problem: with 8 block numbers per inode, the largest a file can be is:

512 * 8 = 4096 bytes (~4KB).

Assuming that the size of an inode is fixed, what can we do?

Singly-indirect addressing

i_addr stores 8 numbers of blocks that contain block numbers, and those block numbers are of blocks that contain payload data.

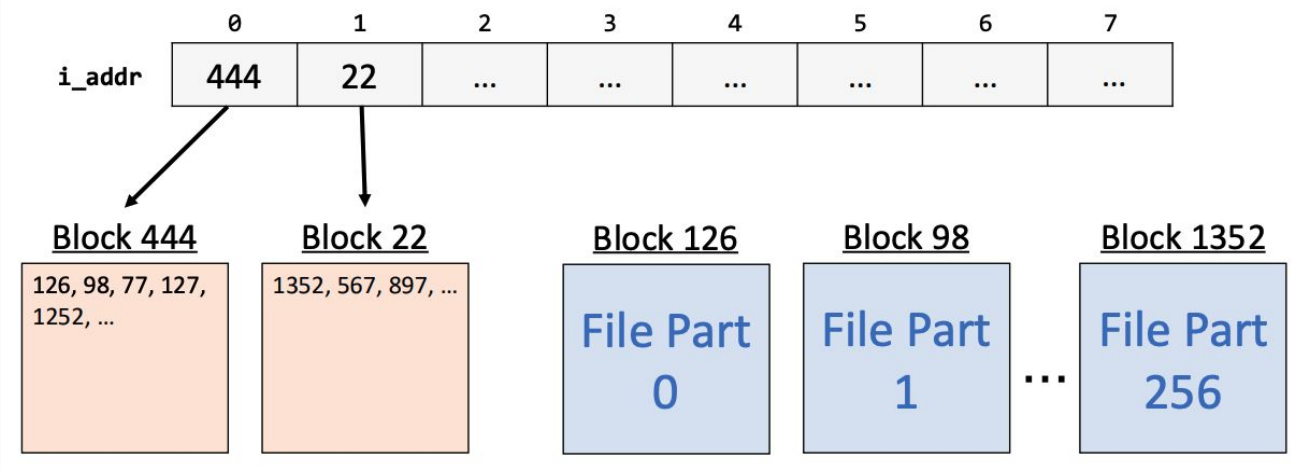


Singly-indirect addressing

Each block has size 512 byte.

Block number is 2-bytes.

Then an **indirect block** can reference 256 other blocks that contain payload data



Large files

So far, i_addr can be used to store:

1. the numbers of blocks that contain **payload data**.
2. the numbers of blocks that contain **other block numbers** that contain **payload data.**

How do we interpret i_addr?

Unix V6 has two inode “modes”: small and large, that dictate how it uses i_addr.

```
if ((inode.i_mode & ILARG) != 0) { // inode is large mode
```

Large files

Unix V6 has two inode “modes”: small and large, that dictate how it uses i_addr.

```
if ((inode.i_mode & ILARG) != 0) {
// inode is large mode
```

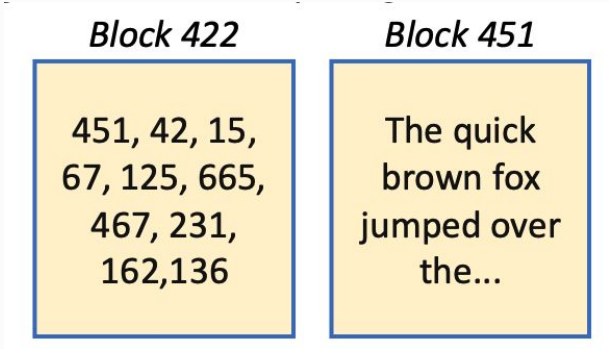
```
struct inode {
  uint16_t  i_mode;     // bit vector of file
                        //   type and permissions
  uint8_t   i_nlink;    // number of references
                        //   to file
  uint8_t   i_uid;      // owner
  uint8_t   i_gid;      // group of owner
  uint8_t   i_size0;    // most significant byte
                        //  of size
  uint16_t  i_size1;    // lower two bytes of size
                        //  (size is encoded in a
                        //  three-byte number)
  uint16_t  i_addr[8];  // device addresses
                        //   constituting file
  uint16_t  i_atime[2]; // access time
  uint16_t  i_mtime[2]; // modify time
};
```

Indirect Addressing: Design Decisions

We could use singly-indirect addressing in other ways, too – all design decisions!

- We could make just some of the block numbers in an inode use singly-indirect addressing, and others still be direct addressing.
- We could have just one "mode" for files, instead of 2.

One argument against indirect addressing: it takes more steps to get to the data and uses more blocks.



Indirect addressing: Practice # 1

Design: 1 block = 1 sector = 512 bytes; block numbers are 2 bytes big

Let's say we have a “large mode” inode with the following information

Inode:

size = 200,000 bytes

i_addr = [56, 122, …]

Which singly-indirect block stores the block number holding the index 150,000th byte of the file? (Hint: 256 * 512 = 131,072)

Indirect addressing

Let's assume for now that an inode for a large file uses all 8 block numbers for singly-indirect addressing. What is the largest file size this supports? Each block number is 2 bytes big.

8 block numbers in an inode x

256 block numbers per singly-indirect block x

512 bytes per block

= ~1MB

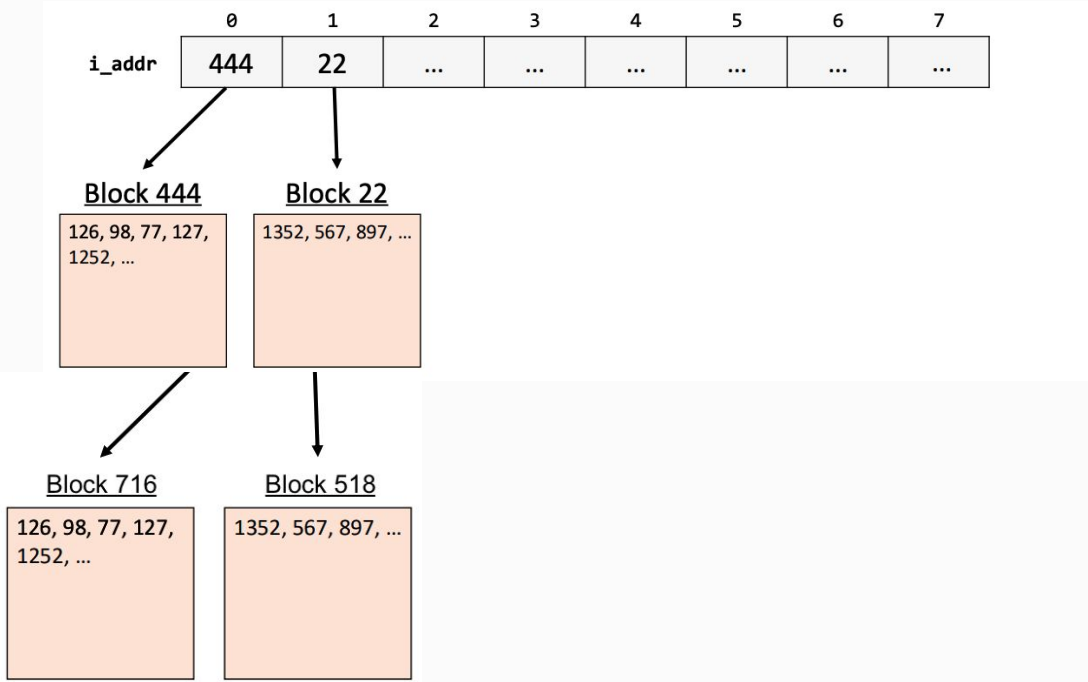
Larger files

Problem: even with singly-indirect addressing, the largest a file can be is 8 * 256 * 512 = 1,048,576 bytes (~1MB).

Assuming that the size of an inode is fixed, what can we do?

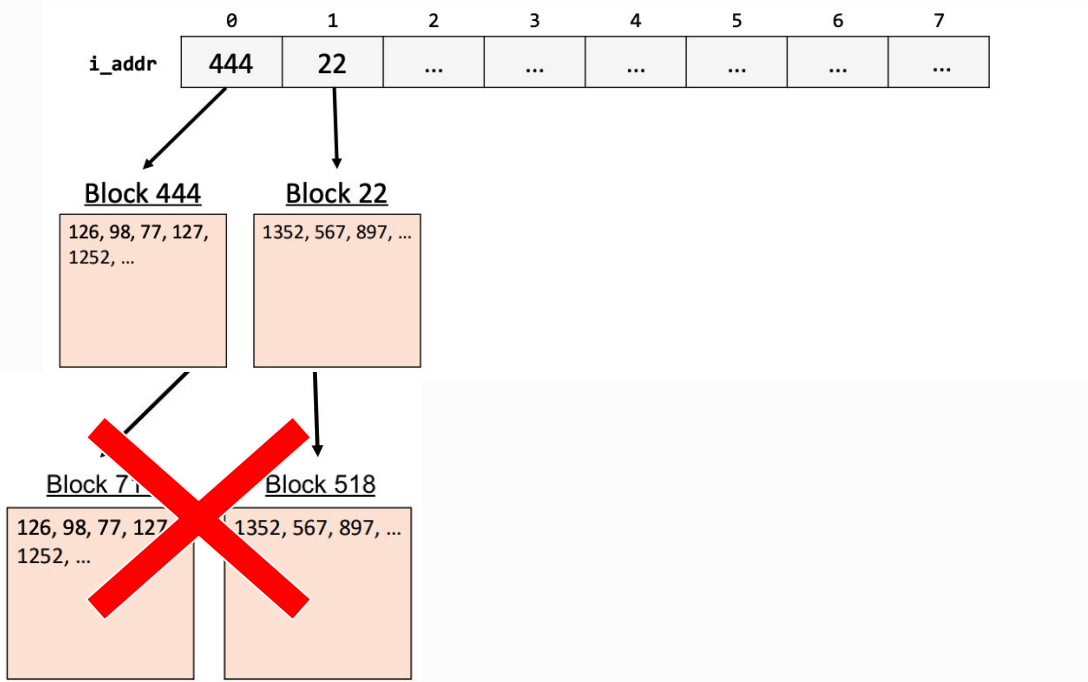
Doubly-indirect addressing

i_addr stores 8 numbers of blocks that contain block numbers, and those block numbers are of blocks that contain payload data.



Doubly-indirect addressing

i_addr stores 8 numbers of blocks that contain block numbers, and those block numbers are of blocks that contain payload data.



Larger files

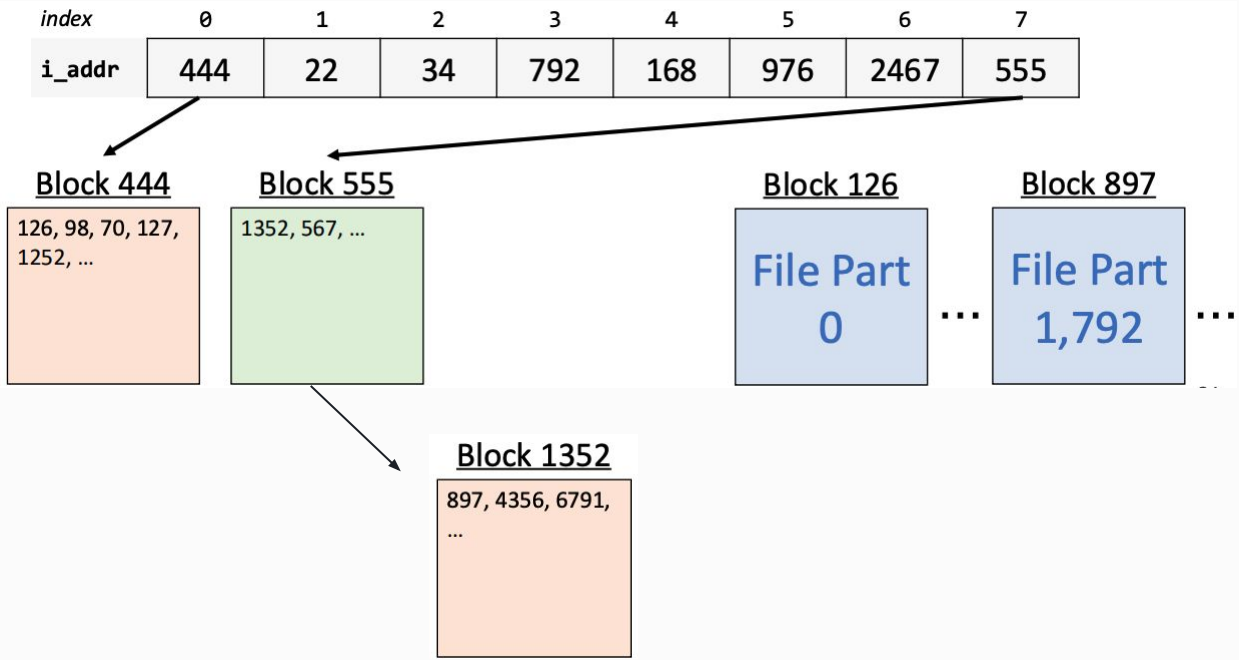
Problem: even with singly-indirect addressing, the largest a file can be is 8 * 256 * 512 = 1,048,576 bytes (~1MB).

Assuming that the size of an inode is fixed, what can we do?

Solution: let's have the 8th entry in i_addr use doubly-indirect addressing; store a block number for a block that contains singly-indirect block numbers.

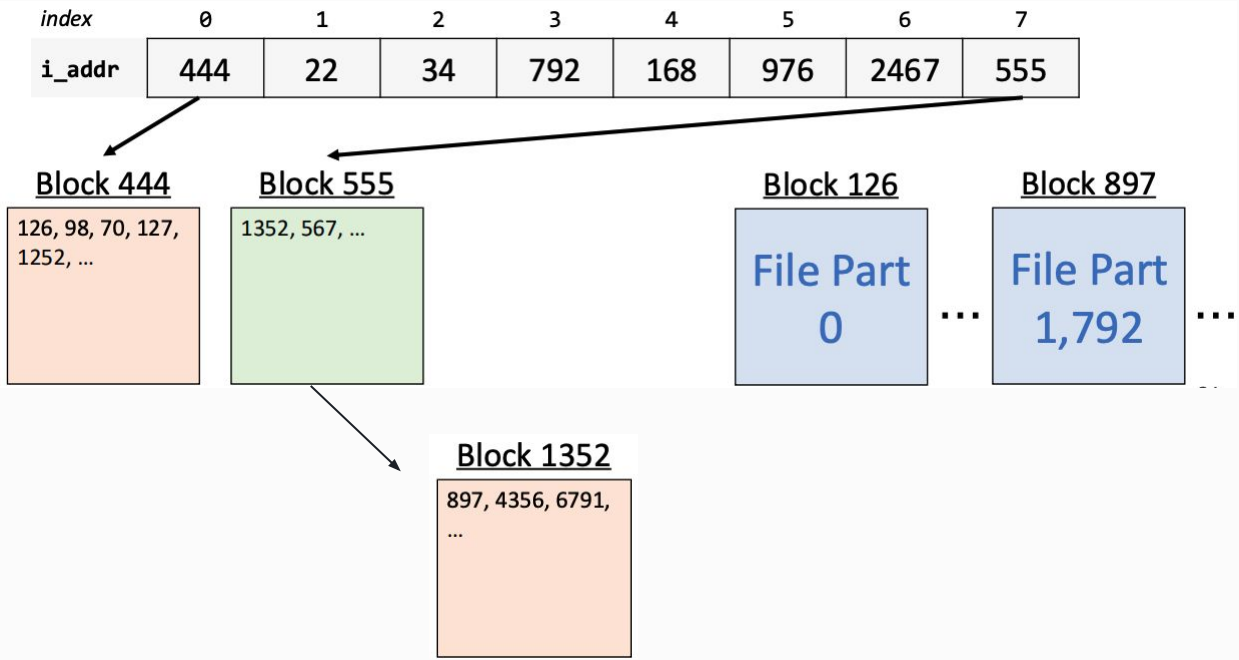
Larger files

If the file is large, the first 7 entries in **i_addr** are singly-indirect block numbers (block numbers of blocks that contain direct block numbers). The 8th entry (if needed) is a doubly-indirect block number (the number of a block that contains singly-indirect block numbers).



Larger files

Another way to think about it: a file can be represented using at most 7 + 256 = 263 singly-indirect blocks. The numbers of the first seven are stored in the inode. The numbers of the remaining 256 are stored in a block whose block number is stored in the inode.



Larger files

An inode for a large file stores 7 singly-indirect block numbers and 1 doubly indirect block number. What is the largest file size this supports? Each block number is 2 bytes big.

(7+256) singly-indirect block numbers total x

256 block numbers per singly-indirect block x

512 bytes per block

= ~34MB

Practice

What is the smallest file size (in bytes) that would require using the doubly indirect block to store its data?

Practice

What is the smallest file size (in bytes) that would require using the doubly indirect block to store its data?

Files up to (7 * 256 * 512) bytes are representable using just the 7 singly indirect blocks. Files of (7 * 256 * 512) + 1 or more bytes would need the doubly-indirect block as well.

Any questions?
No stupid question!

Let's discuss any doubts or concerns.
If something comes up later, contact abdelrahman@aun.edu.eg